

Pebble

Adding agents does not add parallelism.
Adding coordination does.

Four stages · five agents · one dispatch



One dispatch, four stages, no steering

4

stages built and verified

5

seats: 4 models + 1 clock

0

human messages after the dispatch (run 1: 5)

160

commits, every one by a seat

29.8 h

wall-clock, about 9 h of it waiting on session limits

~\$25

Featherless spend; Claude seats on a subscription

Run 2 of the same four-stage build. Run 1 is the comparison on the next slides, including what did not improve.

Five seats, one job each



coordinator
Claude Sonnet

Requirements,
design, the
dependency graph,
routing, close-out



developer
GLM-5.3-Flash

Code and unit
tests, one branch
per review batch



tester
GLM-5.3-Flash

Completeness
review, black-box
acceptance suite,
UI checks



reviewer
Claude Sonnet

Clean-copy review,
screenshots, the
only merger



timekeeper
script, no model

Wakes the
coordinator when a
stage goes quiet

Today the routing is by reasoning load, fixed per seat.

High reasoning (planning, corner cases, merge decisions): Claude Sonnet.

Lower reasoning, high volume (code, tests): a cheaper model, about 9× cheaper than GLM-5.2.

Designed for agents, not for an org chart

10 → 5

seats, after a toy rehearsal

- 3 seats cost \$12.40 for about 1.5 stages: coordination, not capacity, was the bill
- Architect is a responsibility of the coordinator, not a seat
- One developer: a second doubles handoffs and merges
- QA folds into the tester; the ship-and-watch loop was cut

Humans sync often because they forget. Agents don't.

- 1 Many small pull requests
One coherent batch per review, tested as a whole
- 2 “Please prioritise this”
A precondition: dependent work doesn't start until its prerequisite is merged
- 3 Daily standup
A timekeeper that speaks only when the room is quiet

Run 1: a rejection that did not stop the line

19:50

Batch 2 rejected

Two defects, reproduced with an independent probe.

20:04

Batch 3: blocked

“...stacked directly on batch 2's tip. I can't fast-forward without pulling in the unfixed defects.”

22:32

Batch 4: “still blocked”

“I haven't seen a batch-2 fix land yet.”

22:57

Batch 5: “same blocker”

“...as batches 3 and 4.” The coordinator then asked if the pipeline was stalled.

The mandate already said “A rejection takes priority over everything.”

It was not enough. Fix verified at 23:21, 3 h 31 min later, and still not mergeable: a stale branch.

A precondition holds. A priority rule doesn't

1

developer.md

"Start a batch only when every batch in its 'builds on' line is merged into main."

Work is a dependency graph. A batch on top of a rejected batch is simply not ready.

2

developer.md

"One branch per review batch... created from the latest main revision, which must already contain every batch it builds on."

Branches such as developer-s2-b3 replace one shared branch that went stale for everyone.

3

coordinator.md

"Arrange the batches so that at least two can start from the stage's first revision wherever the design allows."

Stage 2 and 3 task lists run two batches in parallel; stages 1 and 4 record why they cannot.

Reviews are batches of 3 to 5 per stage; the developer still commits every task for traceability.

What the room logs show, run 1 vs run 2

Measure	Run 1	Run 2
Human messages after the first dispatch	5	0
Ticks sent while the room was healthy	49 of 65 (75%)	1 of 42 (2%)
Batches stacked on a rejected batch	4 messages, one chain	0
Rejection to fix merged	3 h 31 min, not yet mergeable	7, 14, 15, 15, 58 min
Room message cap (10,000)	Full in stage 4; sends rejected	Largest room: 4,621
Stage handover to a new room	Written by the human	Automated handover.md
Close-out before a stage report	None	Roll call to every seat

Counts are from the exported room logs (UTC); keyword counts where noted in FACTORY.md.

UI quality became evidence, not opinion

Pay someone

To (handle)

bob

Amount

5.00

Decimal amount, e.g. 15.00

Note (optional)

coffee

Visibility

Public

We couldn't confirm this payment — it may not have gone through. Press Pay again without changing anything to check.

Pay

Pebble Home Requests Split Authorisations Ada ada Log out

Wallet

Available to spend

70.00 EUR

Total balance

100.00 EUR

Held 30.00 EUR

Refresh

Pay someone

To (handle)

bob

Amount

15.00

Decimal amount, e.g. 15.00

Request money

From (handle)

bob

Amount

10.00

Decimal amount, e.g. 10.00

Left: phone, 375 px. A lost payment response shows “We couldn't confirm this payment”, and Pay retries with the same key. Right: desktop wallet with a 30.00 hold.

35

screenshots committed, at 375 px and 1280 px, in empty, loading, error and success states

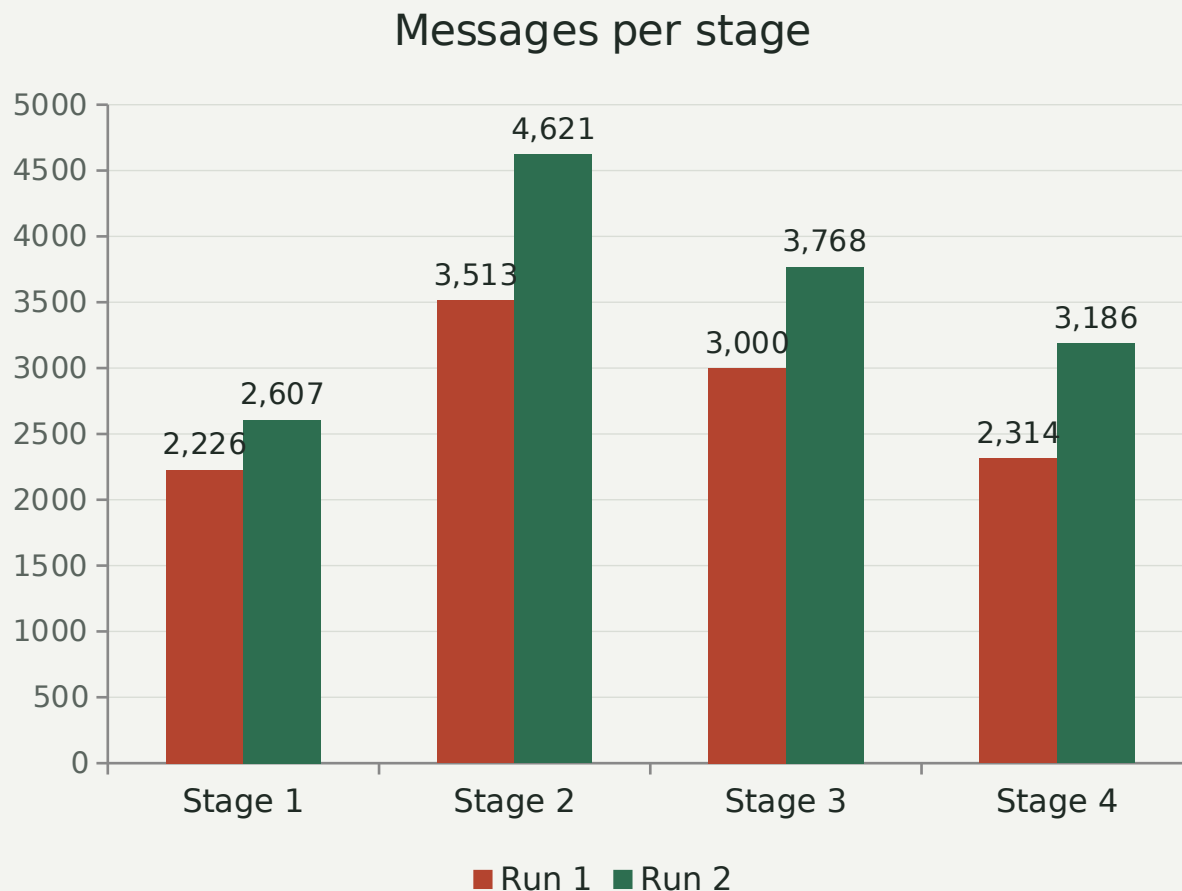
The reviewer must look at them and may reject only against a stated requirement and a screenshot path. One batch was rejected this way and the fix is in the repo.

How the factory catches bad work

Failure	Caught by	Real example from the logs
Contradictory requirement	Tester's completeness review, before code	Stage 4: R303's wording contradicted its own acceptance criterion about who is debited on a refund
Wrong behaviour, tests green	Reviewer's own black-box probes	Stage 1 R79: validation precedence wrong with 112 of 112 unit tests passing
UI defect	Screenshots at two widths	Stage 2: authorization errors rendered in the wrong part of the page
Data integrity	Invariant stress probes	Stage 3 R218: reset never checked the opening balance was non-negative
Crash on real state	Reviewer exercises export with state	Stage 3: export crashed once a statement snapshot existed
Unfinished work	Close-out roll call	Stage 4: a stale RUN.md label caught in verification, fixed before the report

Close-out gate: no stage report until every seat replies “clear”, no branch is left unmerged and every handoff is answered.

What it cost, and what we won't claim



Run 2 spent more, not less

- +28% messages, 25.6 h → 29.8 h, about \$10.50 → \$25
- It bought verification: 35 screenshots, a visual review per UI batch, a close-out per stage, a full handover per room
- Session limits cost about 30% of wall-clock in both runs; seats resumed on their own within about 25 minutes
- The big saving came earlier: GLM-5.2 to GLM-5.3-Flash, about \$8 to \$2.60 per stage

What we'd do next

Route by task

Simple tasks keep the cheaper model; complex ones such as UI go to a stronger one, chosen at run time for developer and tester. Expected: higher cost, less time, better quality. Not yet measured.

Fix the stalls

A failover model for the Claude seats; a timekeeper that reads the limit's reset time (27 of 42 ticks hit an exhausted coordinator); main merged into a branch right before handoff.

Reusable

Mandates name no product, stack or track. Point the factory at a new problem by writing a new dispatch.

github.com/JahnaviSrilekha/payment-app-by-autonomous-agents-hackathon

Run it: `cd stage-4 && PORT=8080 python3 src/server.py`

FACTORY.md has every number and message id in this deck.